HELMSMAN

FACTORY AUTOMATION

DATASHEET

Early release

November 2026 · helmsman.advin.io

The soft PLC you program by describing the machine. The agent writes standard IEC 61131-3 Structured Text from your spec, you check it on a live diagram, and an engineer approves it before it runs.

1. DESCRIBE

State the sequence, the interlocks and the alarms the way you would to a colleague. Attach the drawings.

2. VALIDATE

See the draft as a live diagram and run it against a simulation of your I/O. Tap any output to see what drives it.

3. APPROVE

An engineer signs off with a PIN. The change goes live between scans; every version is kept and can be rolled back.

SPECIFICATIONS

PROGRAMMING	IEC 61131-3 Structured Text: programs, functions and function blocks with instance state, and the standard library of timers, counters, edge detectors, bistables and the numeric, selection and conversion functions. Programs are plain `.st` files you can read, compare between versions and move to another controller.
IMPORTS	PLCopen XML (TC6) from other tools: ladder, FBD and ST bodies lowered to the same Structured Text. Logix L5X exports: the ladder subset (contacts, coils, branches, one-shots, timers, counters, compares, math, moves, subroutine calls); anything outside it is refused by name.
SCAN	Fixed period per program, 1 to 1000 ms. An instruction budget bounds every scan as a watchdog; a fault (division by zero, an index out of range, the budget spent) stops that program alone with its outputs zeroed while the others keep running.
FIELD I/O	EtherNet/IP (CIP) Class 1 scanner: exclusive-owner connections to remote I/O such as the Moxa ioLogik, from a device profile or its EDS file. Modbus TCP master; Modbus RTU over a serial port. Digital I/O on the controller's own header. One named I/O map binds signals to program variables; a bad point marks itself without taking its device offline.
EXISTING PLCS	Appears to them as a standard EtherNet/IP adapter (assemblies, Forward_Open, cyclic I/O). Reads Logix tags from a CompactLogix or ControlLogix and never writes to them: the controller build has no write path.
DATA OUT	OPC UA server for the signals you publish. Webhooks on events. Trends kept on the controller; snapshots of state you can roll back to.
ENGINEERING	A browser editor the controller serves itself: the program as a live function-block diagram with values on the wires, "why" tracing from any output, audited forcing of inputs, a trend, a messages log, go to definition and cross references, familiar keyboard shortcuts. Projects with versions, upload and download, and a Monitor screen for a wall display.
AGENT	Writes and changes programs from a description. Reads what you drop in: a P&ID, an electrical drawing, a photo of the panel, a PDF. Explains any running program in plain words. Its changes arrive as proposals you review before deploy; it reaches the controller only through the same API and policy as a person.
SECURITY	Local accounts with roles and a PIN; an approval policy per controller decides what deploys at once and what waits for an engineer; every change, force and release in an audit log; HTTPS with a certificate the controller makes at first boot, its fingerprint printed at startup.
DEPLOYMENT	One compiled service on 64-bit Linux (arm64 or x86-64), headless, with its web screen built in. No cloud needed to run the machine, no database server, nothing on the workstation: a browser is the whole install. Open source under Apache 2.0.

WHY IT HOLDS UP

ROBUST

A minimal set of dependencies. Every change passes more than 1,300 automated tests and interoperability tests before it ships.

SIMPLE

One controller, one service, its own web screen. No cloud, no database server, no stack of microservices.

SAFE

Written in a memory-safe language. Changes need an engineer's approval, every change is audited, and the safeguards stay out of your way.

RELIABLE

Compiled, native performance on a fixed scan. A watchdog bounds every scan, and a faulted program stops alone with its outputs safe.

WHAT HAS BEEN TESTED

OpENer reference adapter	**verified**
Helmsman's own adapter, same suite	**verified**
Moxa ioLogik E1213, Modbus TCP and EtherNet/IP	**bench pending**
Rockwell CompactLogix, read without writing	**bench pending**

Each verified row is a passing test or a capture; the suite runs on every change against an open-source EtherNet/IP device in a container. Ethernet protocols today; others as customers need them.

WHAT THE AGENT WRITES

```
PROGRAM line
VAR
  s1, s2 : station;
  jam_1 AT %IX0.0 : BOOL;
  alarm AT %QX0.2 : BOOL;
END_VAR
s1(enable := TRUE, jam := jam_1);
s2(enable := TRUE, jam := FALSE);
alarm := s1.alarm OR s2.alarm;
END_PROGRAM
```

Try it with nothing to install: helmsman.advin.io/playground runs a controller in the browser against simulated I/O, with the same editor and diagram.